

Front End Developer Interview Guide

HTML • CSS • JavaScript • DOM • Async JS • Data Structures

Ek complete, beginner-friendly, in-depth guide
taaki kal interview mein confidence ke saath jaa sako.

All the best! 🍀

Table of Contents

1. HTML Basics — Structure, Semantic Tags, Forms
2. CSS Basics — Box Model, Selectors, Specificity, Positioning
3. CSS Layouts — Flexbox, Grid, Responsive Design
4. JavaScript Fundamentals — Variables, Data Types, Functions
5. Scope, Closures, and the "this" Keyword
6. Objects and Arrays in JavaScript
7. DOM Manipulation and Events
8. Asynchronous JavaScript — Callbacks, Promises, Async/Await
9. Basic Data Structures — Arrays, Maps, Sets
10. Developer Tools & Debugging Tips
11. Common Interview Questions (Quick Answers)
12. Night-Before Cheat Sheet

1. HTML Basics

HTML (HyperText Markup Language) webpage ka **structure/skeleton** banana hai. Har element ek tag ke andar hota hai, jo browser ko batata hai "yeh content kya hai" (heading, paragraph, image, etc.).

Basic HTML Document Structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Page</title>
</head>
<body>
  <h1>Hello World</h1>
  <p>This is a paragraph.</p>
</body>
</html>
```

Yaad rakho: `<!DOCTYPE html>` browser ko batata hai ki yeh HTML5 document hai. `viewport` meta tag mobile responsiveness ke liye zaroori hai.

Semantic HTML Tags

Semantic tags ka matlab hai jo tags apne content ka meaning khud batate hain — sirf `div`/`span` use karne se better hota hai kyunki accessibility aur SEO improve hota hai.

Tag	Use
<code><header></code>	Page ya section ka top part (logo, nav)
<code><nav></code>	Navigation links
<code><main></code>	Page ka main content
<code><section></code>	Related content ka group
<code><article></code>	Independent, self-contained content (blog post)
<code><aside></code>	Sidebar / related but separate content
<code><footer></code>	Page ka bottom part

Common Tags & Attributes

```

<a href="https://example.com" target="_blank">Visit Site</a>
<ul>
  <li>Item 1</li>
  <li>Item 2</li>
</ul>
<table>
  <tr><th>Name</th><th>Age</th></tr>
  <tr><td>Ravi</td><td>22</td></tr>
</table>
```

HTML Forms (Important — always asked)

```
<form action="/submit" method="POST">
  <label for="name">Name:</label>
  <input type="text" id="name" name="name" required>

  <label for="email">Email:</label>
  <input type="email" id="email" name="email">
```

```
<label for="gender">Gender:</label>
<select id="gender" name="gender">
  <option value="male">Male</option>
  <option value="female">Female</option>
</select>

<button type="submit">Submit</button>
</form>
```

label ka for attribute input ke id se match hona chahiye — click karne pe input focus hoga, aur yeh accessibility ke liye best practice hai.

Interview Tip: "Difference between <div> and ?" — div block-level hai (apni line leta hai), span inline hai (sirf apne content jitni jagah leta hai). Yeh classic question hai, zaroor pucha jaata hai.

2. CSS Basics — Box Model, Selectors, Specificity, Positioning

CSS Kaise Likhein

```
/* Inline */
<p style="color:red;">Text</p>

/* Internal - inside <style> tag in <head> */
<style> p { color: red; } </style>

/* External (best practice) */
<link rel="stylesheet" href="style.css">
```

The Box Model (Very Important!)

Har HTML element ek box hota hai jisme 4 layers hoti hain, andar se bahar:

1. **Content** — actual text/image
2. **Padding** — content ke around ki empty space (andar ki side)
3. **Border** — padding ke around ki line
4. **Margin** — border ke bahar ki empty space (elements ke beech ka gap)

```
.box {
  width: 200px;
  padding: 20px;
  border: 5px solid black;
  margin: 10px;
  box-sizing: border-box; /* IMPORTANT */
}
```

box-sizing: border-box use karo — isse width mein padding aur border already included ho jaata hai, total size predictable rehta hai. Default (content-box) mein padding/border width ke *alag se add* hota hai, jo confusion create karta hai.

Selectors

Selector	Example	Matlab
Element	<code>p { }</code>	Sab <code><p></code> tags
Class	<code>.box { }</code>	<code>class="box"</code> wale elements
ID	<code>#header { }</code>	<code>id="header"</code> wala element (unique)
Descendant	<code>div p { }</code>	div ke andar wale p (kitni bhi deep)
Child	<code>div > p { }</code>	div ka <i>direct</i> child p
Attribute	<code>input[type="text"]</code>	Specific attribute wale elements
Pseudo-class	<code>a:hover { }</code>	Element ki state (hover, focus, etc.)
Pseudo-element	<code>p::before { }</code>	Element ke andar virtual content add karna

Pseudo-classes vs Pseudo-elements (common question)

Q: Difference between pseudo-class and pseudo-element?

Pseudo-class (single colon `:`) element ki **state/condition** select karta hai — jaise `:hover`, `:first-child`, `:nth-child(2)`.

Pseudo-element (double colon `::`) element ke andar ek **naya virtual part** banata hai — jaise `::before`, `::after`, `::first-letter`.

Specificity (Zaroor Pucha Jaata Hai)

Jab multiple CSS rules same element pe apply hote hain, browser decide karta hai kaunsa rule "win" karega specificity ke basis pe:

Inline style > 1000 points

```
ID selector          > 100 points
Class/Attribute/Pseudo-class > 10 points
Element/Pseudo-element > 1 point
```

Example: #nav .link (ID + class) hamesha .link (sirf class) ko beat karega, chahe order jo bhi ho. !important sabse strong hai lekin isse avoid karna best practice hai.

Positioning

Value	Behavior
static	Default — normal document flow mein rehta hai
relative	Apni normal position se offset hota hai (top/left etc.), lekin space reserve rehti hai
absolute	Document flow se nikal jaata hai; nearest positioned parent ke relative position hoti hai
fixed	Viewport ke relative fixed — scroll karne pe bhi nahi hilti
sticky	Normal flow mein rehta hai jab tak scroll threshold cross na ho, phir "stick" ho jaata hai

Interview Tip: "absolute" element ka reference point uska nearest ancestor hota hai jiska position relative/absolute/fixed ho. Agar koi na ho, toh <html> element ke relative position hoti hai. Yeh concept diagram bana ke explain karna best hai.

3. CSS Layouts — Flexbox, Grid, Responsive Design

Flexbox (1-Dimensional Layout)

Flexbox items ko ek row ya column mein arrange karne ke liye best hai.

```
.container {
  display: flex;
  flex-direction: row;      /* row | column */
  justify-content: center;  /* main-axis alignment */
  align-items: center;      /* cross-axis alignment */
  flex-wrap: wrap;
  gap: 10px;
}
.item {
  flex: 1; /* grow, shrink, basis shorthand */
}
```

Property	Kaam
<code>justify-content</code>	Main axis pe alignment (flex-start, center, space-between, space-around)
<code>align-items</code>	Cross axis pe alignment (flex-start, center, stretch)
<code>flex-grow</code>	Extra space ko kitna lena hai
<code>flex-shrink</code>	Space kam ho toh kitna shrink hona hai

CSS Grid (2-Dimensional Layout)

```
.container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-template-rows: auto auto;
  gap: 16px;
}
.item {
  grid-column: span 2; /* 2 columns occupy karega */
}
```

Flexbox vs Grid: Flexbox 1D hai (ek row YA ek column), Grid 2D hai (rows AND columns ek saath control). Simple use case ke liye Flexbox, complex page layout ke liye Grid.

Responsive Design (Media Queries)

```
/* Mobile-first approach */
.container {
  display: block;
}

@media (min-width: 768px) {
  .container {
    display: flex;
  }
}

@media (max-width: 600px) {
  .sidebar {
    display: none;
  }
}
```

Fluid layout banane ke liye fixed px ki jagah %, vw/vh, rem/em, aur flex/grid use karo. Isse layout different screen sizes pe automatically adjust ho jaata hai.

Useful Shorthands

```
margin: 10px 20px;           /* top-bottom | left-right */
padding: 10px 20px 15px 5px; /* top | right | bottom | left */
border: 1px solid black;      /* width | style | color */
background: #fff url(img.png) no-repeat center;
font: italic bold 16px/1.5 Arial, sans-serif;
```

4. JavaScript Fundamentals

Variables: var, let, const

Keyword	Scope	Re-declare?	Re-assign?	Hoisting
var	Function-scoped	Yes	Yes	Hoisted, initialized as <code>undefined</code>
let	Block-scoped	No	Yes	Hoisted but in "temporal dead zone"
const	Block-scoped	No	No	Same as let

Aaj kal `let` aur `const` use karte hain, `var` avoid karte hain (purana, buggy behavior deta hai block scoping mein). Default `const` use karo, jab value change karni ho tab hi `let`.

Data Types

```
// Primitive types
let num = 25;
let str = "Hello";
let bool = true;
let nothing = null;
let notDefined = undefined;
let sym = Symbol("id");
let big = 123n; // BigInt

// Reference type
let arr = [1,2,3];
let obj = { name: "Ravi" };

typeof num;    // "number"
typeof str;    // "string"
typeof obj;    // "object"
```

Functions and Their Types

```
// 1. Function Declaration (hoisted, can call before defined)
function greet(name) {
  return "Hello " + name;
}

// 2. Function Expression (NOT hoisted)
const greet2 = function(name) {
  return "Hi " + name;
};

// 3. Arrow Function (shorter syntax, no own 'this')
const greet3 = (name) => {
  return "Hey " + name;
};
const square = n => n * n; // implicit return

// 4. IIFE - Immediately Invoked Function Expression
(function() {
  console.log("Runs immediately!");
})();

// 5. Default parameters
function multiply(a, b = 2) {
  return a * b;
}

// 6. Rest parameters
function sum(...nums) {
  return nums.reduce((a, b) => a + b, 0);
}
```

```
}
```

Interview Tip: Arrow functions apna khud ka `this` nahi banate — woh surrounding (lexical) scope se `this` le lete hain. Regular functions ka apna `this` hota hai jo *call karne ke tareeke* pe depend karta hai. Yeh ek favourite interview question hai.

5. Scope, Closures, and "this" Keyword

Scope

```
let globalVar = "I'm global";

function outer() {
  let outerVar = "I'm in outer";
  if (true) {
    let blockVar = "I'm block scoped"; // only inside this {}
    console.log(globalVar, outerVar, blockVar); // works
  }
  // console.log(blockVar); // ERROR - not accessible here
}
```

Closures (Bahut Important Concept)

Closure tab hota hai jab ek inner function apne outer function ke variables ko "yaad" rakhta hai, chahe outer function execution khatam ho chuka ho.

```
function counter() {
  let count = 0; // this is "closed over"
  return function() {
    count++;
    return count;
  };
}

const increment = counter();
console.log(increment()); // 1
console.log(increment()); // 2
console.log(increment()); // 3
// 'count' private hai, bahar se directly access nahi ho sakta
```

Closures ka real use: **data privacy** (private variables banana), function factories, aur event handlers/callbacks mein state yaad rakhna.

"this" Keyword

Context	"this" refers to
Regular function (global call)	<code>undefined</code> (strict mode) ya <code>window</code>
Object method	Jo object method ko call kar raha hai
Arrow function	Surrounding lexical scope ka <code>this</code>
Constructor (with <code>new</code>)	Newly created object
<code>call()</code> / <code>apply()</code> / <code>bind()</code>	Manually specify kiya gaya object

```
const person = {
  name: "Amit",
  greet: function() {
    console.log("Hi, I'm " + this.name); // works: this = person
  },
  greetArrow: () => {
    console.log("Hi, I'm " + this.name); // 'this' yahan object nahi hai!
  }
};

person.greet(); // "Hi, I'm Amit"
person.greetArrow(); // "Hi, I'm undefined"
```

6. Objects and Arrays in JavaScript

Objects

```
const student = {
  name: "Priya",
  age: 21,
  skills: ["HTML", "CSS", "JS"],
  greet() {
    console.log(`Hi, I'm ${this.name}`);
  }
};

// Accessing
console.log(student.name);    // dot notation
console.log(student["age"]);  // bracket notation

// Adding / Updating
student.city = "Delhi";
student.age = 22;

// Deleting
delete student.city;

// Looping
for (let key in student) {
  console.log(key, student[key]);
}

// Useful methods
Object.keys(student);    // ["name","age","skills","greet"]
Object.values(student);
Object.entries(student);

// Spread & destructuring
const { name, age } = student;
const copy = { ...student, city: "Mumbai" };
```

Arrays and Common Array Methods (Interview mein bahut aate hain)

```
const nums = [1, 2, 3, 4, 5];

// map - transform each element, returns NEW array
const doubled = nums.map(n => n * 2); // [2,4,6,8,10]

// filter - keep elements matching condition
const evens = nums.filter(n => n % 2 === 0); // [2,4]

// reduce - accumulate into single value
const total = nums.reduce((sum, n) => sum + n, 0); // 15

// forEach - just loop, no return
nums.forEach(n => console.log(n));

// find - first matching element
const found = nums.find(n => n > 3); // 4

// includes / indexOf
nums.includes(3); // true

// sort
[3,1,2].sort((a,b) => a - b); // [1,2,3]

// spread to copy/merge
const arr2 = [...nums, 6, 7];
```

```
// destructuring
const [first, second] = nums;
```

Interview Tip: `map` hamesha same length ka naya array return karta hai. `filter` condition ke basis pe elements chhod deta hai. `reduce` sabse powerful hai — sum, max, grouping, flattening sab kar sakta hai. Yeh teeno bahut common coding questions mein use hote hain.

7. DOM Manipulation and Events

Selecting Elements

```
document.getElementById("title");
document.getElementsByClassName("item"); // HTMLCollection
document.getElementsByTagName("p");
document.querySelector(".item"); // first match
document.querySelectorAll(".item"); // all matches (NodeList)
```

Changing Content and Styles

```
const el = document.querySelector("#title");
el.textContent = "New Text"; // only text
el.innerHTML = "<b>Bold</b>"; // can insert HTML (careful - XSS risk)
el.style.color = "blue";
el.classList.add("active");
el.classList.remove("hidden");
el.classList.toggle("open");
el.setAttribute("data-id", "5");
```

Creating and Removing Elements

```
const li = document.createElement("li");
li.textContent = "New item";
document.querySelector("ul").appendChild(li);

const oldEl = document.querySelector(".old");
oldEl.remove();
```

Events and Event Handling

```
const btn = document.querySelector("#myBtn");

btn.addEventListener("click", function(event) {
  console.log("Button clicked!");
  console.log(event.target); // which element triggered it
});

// Event delegation - efficient way to handle many child elements
document.querySelector("ul").addEventListener("click", function(e) {
  if (e.target.tagName === "LI") {
    console.log("Clicked on:", e.target.textContent);
  }
});
```

Event Delegation: Har child element pe alag listener lagane ke bajaye, parent pe ek listener lagao aur `e.target` se check karo kaunsa child click hua. Yeh performance ke liye best practice hai, especially dynamically added elements ke liye.

Handling Forms with JS

```
const form = document.querySelector("#myForm");

form.addEventListener("submit", function(e) {
  e.preventDefault(); // page reload rokta hai
  const name = document.querySelector("#name").value;
  if (name.trim() === "") {
    alert("Name required!");
    return;
  }
});
```

```
console.log("Submitted:", name);  
});
```

Event Bubbling vs Capturing

Q: Bubbling vs Capturing kya hai?

Jab kisi element pe event trigger hota hai, toh **capturing phase** mein event top (document) se target tak aata hai, aur **bubbling phase** mein target se wapas top tak jaata hai. Default: JS bubbling phase use karta hai. Capturing use karne ke liye

```
addEventListener(type, fn, true).
```

8. Asynchronous JavaScript

setTimeout & setInterval

```
setTimeout(() => {
  console.log("Runs once after 2 seconds");
}, 2000);

const id = setInterval(() => {
  console.log("Runs every 1 second");
}, 1000);

clearInterval(id); // stop it
```

Callbacks (Purana Tareeka)

```
function fetchData(callback) {
  setTimeout(() => {
    callback("Data received");
  }, 1000);
}

fetchData((data) => console.log(data));
// Problem: bahut saare nested callbacks = "Callback Hell"
```

Promises (Better Way)

Promise ek object hai jo represent karta hai future mein aane wali value ko. Iski 3 states hoti hain: **pending**, **fulfilled**, **rejected**.

```
const promise = new Promise((resolve, reject) => {
  const success = true;
  setTimeout(() => {
    if (success) resolve("Data loaded!");
    else reject("Error occurred!");
  }, 1000);
});

promise
  .then(result => console.log(result))
  .catch(error => console.log(error))
  .finally(() => console.log("Done"));
```

Async / Await (Modern & Cleanest Way)

```
function fetchData() {
  return new Promise(resolve => {
    setTimeout(() => resolve("Data!"), 1000);
  });
}

async function getData() {
  try {
    console.log("Loading...");
    const result = await fetchData(); // waits without blocking
    console.log(result);
  } catch (error) {
    console.log("Error:", error);
  }
}

getData();
```

async function hamesha ek Promise return karta hai. await sirf async function ke andar use ho sakta hai, aur woh Promise resolve hone tak "wait" karta hai (bina baaki page block kiye).

Fetch API (Real World Example)

```
async function getUsers() {  
  const response = await fetch("https://api.example.com/users");  
  const data = await response.json();  
  console.log(data);  
}
```

Interview Tip: "Callback vs Promise vs Async-Await?" — Sab async code handle karne ke tareeke hain, evolution isi order mein hua: Callbacks → Promises (chaining se better readability) → Async/Await (synchronous jaisa dikhta hai, sabse readable). Async/Await internally Promises hi use karta hai.

9. Basic Data Structures

Arrays

```
const arr = [10, 20, 30];
arr.push(40);      // add at end
arr.pop();          // remove from end
arr.unshift(5);     // add at start
arr.shift();        // remove from start
arr.slice(1, 3);    // returns NEW array (non-mutating)
arr.splice(1, 1);   // MODIFIES original array (removes/inserts)
arr.length;        // 3
```

Map (Key-Value Pairs, better than Object for this use)

```
const map = new Map();
map.set("name", "Ravi");
map.set("age", 25);

map.get("name");    // "Ravi"
map.has("age");     // true
map.delete("age");
map.size;           // 1

for (let [key, value] of map) {
  console.log(key, value);
}
```

Map vs Object: Map keys *koi bhi* type ho sakti hain (object bhi), order guaranteed rehta hai, aur size directly milta hai. Object mein keys sirf string/symbol hoti hain.

Set (Unique Values Only)

```
const set = new Set([1, 2, 2, 3, 3, 3]);
console.log(set); // Set {1, 2, 3} - duplicates removed automatically

set.add(4);
set.has(2);      // true
set.delete(1);
set.size;        // 3
```

10. Developer Tools & Debugging

- **Elements tab:** Live HTML/CSS dekh aur edit kar sakte ho.
- **Console tab:** `console.log()` se values check karo, errors yahan dikhte hain.
- **Network tab:** API calls, request/response, loading time dekho.
- **Sources tab:** Breakpoints laga ke JS code step-by-step debug karo.
- **Debugger keyword:** Code mein `debugger;` likh do, execution wahin pause ho jaayega jab DevTools open ho.

```
// Quick debugging techniques
console.log("value:", x);
console.table(arrayOfObjects); // nicely formatted table
console.error("Something went wrong");
debugger; // pauses execution here in DevTools
```

11. Common Interview Questions (Quick Answers)

Q1: == vs === difference?

== loose equality hai (type conversion karta hai compare karne se pehle). === strict equality hai (type bhi match hona chahiye). Hamesha === use karo.

Q2: null vs undefined?

undefined matlab variable declare hua hai but value assign nahi hui. null ek intentional "empty value" hai jo developer khud assign karta hai.

Q3: Event loop kya hai?

JavaScript single-threaded hai. Event loop check karta rehta hai ki call stack khali hai ya nahi, aur tab callback queue se tasks (jaise setTimeout callbacks) ko stack mein daal deta hai. Isi wajah se async code non-blocking hota hai.

Q4: What is hoisting?

JS execution se pehle variable aur function declarations ko memory mein "top" pe move kar deta hai (conceptually). var aur function declarations hoist hote hain fully; let/const hoist toh hote hain but temporal dead zone mein rehte hain.

Q5: What is a pure function?

Jo function same input ke liye hamesha same output de, aur koi external state modify na kare (no side effects).

Q6: Difference between let and var scoping - example?

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 100);  
}  
// Output: 3 3 3 (var is function-scoped, same 'i' shared)  
  
for (let i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 100);  
}  
// Output: 0 1 2 (let creates new 'i' for each iteration)
```

Q7: How would you center a div?

```
.parent {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  height: 100vh;  
}
```

Yeh sabse common aur modern tareeka hai.

Q8: What is CSS specificity conflict resolve karne ka order?

Inline > ID > Class/Attribute/Pseudo-class > Element. Same specificity ho toh *last defined rule* jeetega.

12. Night-Before Cheat Sheet <

Kal subah interview se pehle yeh points ek baar zaroor daga lo:

- Box model: content → padding → border → margin, aur `box-sizing: border-box` ka use.
- Specificity order: inline > ID > class > element.
- Flexbox = 1D, Grid = 2D.
- let/const block-scoped, var function-scoped — avoid var.
- Arrow functions ka apna `this` nahi hota.
- Closures = inner function outer variables yaad rakhta hai.
- map/filter/reduce — teeno ka use case clearly bata sako.
- Async/Await Promises ke upar syntactic sugar hai.
- Event delegation = parent pe listener, e.target se check.
- === hamesha use karo, == nahi.

General Tips for Interview:

- 1) Jab bhi coding problem mile, pehle *zor se socho aur approach explain karo*, seedha code mat likhna shuru karo.
- 2) Agar kuch nahi pata, honestly bolo "mujhe exact nahi pata, but main aise approach karunga..." — guess karne se better hai.
- 3) Apna code explain karna practice karo — "main yeh isliye kar raha hoon kyunki..."
- 4) Calm raho, deep breath lo, aur confidently baat karo. Basics clear hain toh tum ready ho!

All the best for tomorrow! Tum kar loge. 🍀